PANADERO

System Overview and Requirements

PANADERO Baking System — Local Bakeshop Production and Pre-Order System

University of Cabuyao • College of Computing Studies • Version 1.0 • 12 October 2026

System Overview (one-page summary)

Problem statement

A small bakery handles walk-in product availability, customer pre-orders, reservations, and daily baking stock. When these records are kept in separate notes or informal messages, staff can miss requests, repeat data entry, or give customers inconsistent information about stock and order status.

Solution and system boundary

PANADERO is a browser-based bakery ordering and management system with customer, staff, and administrator portals. The frontend communicates with a Node.js/Express API using JSON; the backend reads and writes the relational MySQL/MariaDB database. Customers browse the catalog, submit orders or scheduled reservations, and check their request history. Staff record stock and fresh batches and process request statuses. Administrators maintain shared catalog, account, and stock information. The system does not process online card payments; checkout records a selected payment method only.

Target users

User	Primary use
Customer	Browse products; register or sign in; prepare an order or reservation; review statuses and notifications.
Bakery staff	Review operational summaries; maintain stock and baking batches; process orders and reservations.
Administrator	Maintain categories, products, user records, stock, and the administrative overview.

System objectives

- Present bakery products, category details, prices, and current availability in one customer-facing catalog.
- Accept and reference customer orders and scheduled reservations; keep related stock changes consistent.
- Give staff a shared workspace for inventory, production batches, and request status updates.
- Restrict customer, staff, and administrator functions according to their roles.

Architecture at a glance: Customer / Staff / Admin browser pages → HTTP/JSON → Express API → MySQL/MariaDB (mysql2). Database credentials remain on the server; the browser never connects directly to SQL.

Functional Requirements

The requirements below describe the current source-code workflows. Expected outputs represent the successful path. Error responses are returned for invalid data, unauthorized roles, unavailable stock, or unavailable database connections.

ID	Function / description	Authorized user	Expected system output
FR-01	Account access: register, sign in/out, and capture password-reset requests.	Visitor / Customer	A customer session or a clear validation/error response. Passwords are stored as hashes; reset requests return a generic message.
FR-02	Catalog: view categories and products, search, filter, and review stock availability.	Visitor / Customer	Product cards with names, category, image, price, and availability.
FR-03	Cart: add products, change quantities, remove items, and review totals.	Customer / Visitor	Updated cart contents and subtotal in browser storage.
FR-04	Order submission: validate product IDs, quantities, current prices, and stock.	Customer	Order reference and persisted order header/lines; stock is decremented within a transaction.
FR-05	Reservation: request a product quantity and preferred date/time.	Customer	Reservation reference and saved schedule/status; input errors are reported.
FR-06	Customer activity: list the signed-in customer's orders, reservations, and completion notices.	Customer	Request cards/statuses and notification read/unread state scoped to the signed-in user.
FR-07	Staff dashboard: summarize stock and incoming / ready requests.	Staff	Operational metric cards and recent request rows.
FR-08	Inventory and batches: update nonnegative stock and record baked/expiry dates and notes.	Staff	Saved stock quantity and inventory-batch history; a batch adds quantity to stock.
FR-09	Request workflow: advance order/reservation status using allowed transitions.	Staff; limited Admin actions	Persisted new status; invalid transitions are rejected.
FR-10	Administration: manage categories, products, users, stock, and view request summaries.	Administrator	Updated admin view and persisted supported edits; status changes use validated routes.
FR-11	Operations checks: expose build information and database health.	Maintainer / Operations	Build identifier and health response; unavailable database is reported as unavailable.

Non-Functional Requirements

The current design addresses the following quality attributes. Where measurements or deployment controls were not available, that verification limit is stated rather than assumed.

Quality attribute	How the current system addresses it / verification limit
Usability	Role-specific navigation separates customer, staff, and admin tasks. Catalog categories/search, visible statuses, confirmation messages, and responsive page styles support common workflows. No formal usability study was supplied.
Performance	The application uses focused JSON endpoints and a MySQL connection pool. No load test, concurrency test, or response-time benchmark was run; production-scale performance remains unverified.
Reliability	InnoDB tables and foreign keys protect relational integrity. Order creation and batch recording use transactions to commit related writes together or roll them back. Runtime reliability still depends on a correctly configured and running database.
Security	Passwords are hashed with bcrypt; protected API routes use bearer-token authentication and role checks; parameterized SQL is used for key writes; customer history is scoped by account. Before public deployment, review in-memory sessions, CORS scope, HTTPS, rate limiting, and the demo accounts.
Maintainability	Role-specific frontend folders, shared API/service scripts, npm scripts, the SQL schema, setup notes, and source documentation provide a maintainable baseline. The backend has concentrated logic in server.js and some legacy folder names are inconsistent, so future refactoring should preserve referenced paths.

VERIFICATION STATUS

Source syntax and selected HTTP routes were smoke-checked. A MySQL service was unavailable during documentation validation, so database-backed workflows and the SQL query demonstrations were not executed against a live database.